Batch-invariant kernels on Apple's M5 GPU and its matrix units

An exact speculative-decoding reference for a streamed 125B mixture-of-experts model, and what MLX 0.32.2 computes at each row count.

Kamen Levi
Independent researcher

9 November 2026

Abstract

Speculative decoding is exact only if verifying k drafted tokens in one batched call produces the same bits as decoding them one at a time. On Apple's M5 GPU with MLX 0.32.2 it does not. The framework picks a different kernel for 1, 2-8, 16 and thousands of rows (vector kernels, split-K kernels, matrix-unit GEMMs), and each rounds differently. With the stock kernels, all 16 rows of a verify pass already differ from one-token decoding at the first matmul of a 125B mixture-of-experts model [K].

We build a batch-invariant kernel set for that model (48 layers, 512 experts per layer, 10 per token; the expert store runs at about 3.2 bits per expert weight):

- dense projections padded to one 16-row shape, on a matrix-unit kernel that multiplies the raw codes;
- routed experts on one 16-row matrix-unit tile per expert, then a fixed split-K over a record layout native to the matrix unit's fragments;
- a hyper-connection read that replicates MLX's own split-K loop bit for bit;
- attention pinned to fixed key buckets and length classes.

Under it, 16 one-token steps and one 16-row verify agree on all 37 records of a forward through a 12-layer slice [K], and full-length audits agree on every kept row ([MAIN] section 3.2). The dense part of a check falls from 56.8 to 27.4 ms [K]. On the expert path the matrix-unit kernel beats our ALU kernel by x1.062 in whole decode in an engine A/B [K]. Batching a prompt's MoE layers over all chunks is exact once two kernels are pinned, and prefills 1.6K-7.9K-token prompts x1.14-1.21 faster [K]. We map which MLX kernels are row-invariant at decode and prefill widths, and report ten MLX findings with reproductions, among them a sorted-gather bug that leaves rows unwritten past 32,767 rows (independently of its upstream report) and stale reads of CPU-written memory inside one command buffer.

Where we lose. One-token decoding through the padded kernels runs at x0.75 of the stock kernels [K]. Our exact expert kernel runs at 70-74 % of the rate its own loads stream at [K]. The exact batched prefill peaks at 46.2 GB, over the 45 GB line [K]. And on the same machine and the same 48 prompts, against Strata (MIT; github.com/Niko1221/Strata), our decode is ahead on code and math and about level on chat, exact chat decode is behind, and Strata's first token (0.5-1.0 s) beats ours (about 2.7-4.6 s), so up to 256 generated tokens Strata wins or ties every whole-request number ([MAIN] Appendix G has the table).

---

1. Introduction

1.1 Why exactness needs batch invariance

- One-token greedy decoding defines the reference output. A speculative check computes 16 positions in one call. If a kernel's result for one row depends on how many rows share the call, the check's argmax can flip at a near-tie, and the output drifts from the reference.
- The same holds for prefill. A prompt processed in 1,024-token chunks is the reference; a faster prefill that puts more rows into each call must reproduce the chunked bits, or the first decoded tokens move.
- MLX chooses kernels by row count (section 2.2), so stock MLX gives neither property on this model (section 3).

1.2 Contributions

1. **An invariance map of MLX 0.32.2 on the M5** (section 3): for each op class of this model, whether a row's bits depend on the call's row count, at decode and verify widths (1-16 rows) and at prefill widths (1K-8K rows). Every entry is measured.

2. **A batch-invariant kernel set** (section 4) on the GPU's SIMD units and its matrix units, with what each kernel costs or saves.
3. **MLX findings** (section 5), each with a minimal reproduction script and its workaround.
4. **GPU and matrix-unit measurements** (section 6): peak, arithmetic order, streaming efficiency, dispatch cost, and where a check's time goes.
5. **Negative results** (section 7) and **what we left on the table** (section 9).

1.3 What we do not claim

- Not a new principle. Batch-invariant kernels with a fixed reduction order per element are published (Thinking Machines Lab, section 8). Ours is a port of the idea to Metal, MLX and the M5's matrix units, at this model's shapes, with the speculative check as the consumer.
- Not exact to the original weights, and not exact across references. "Exact (to R)" means output tokens, and the logits row behind every kept token, are bit-identical to one-token greedy decoding under reference R ([MAIN] section 3.1). At 45 GB, R is NAX6-N.
- Not a whole-system speed claim. Speeds here are per kernel or per check unless a row says otherwise; the system's whole decode, whole request and first token live in [MAIN] section 6.
- Not first. Two MLX findings were found before us: the sorted-gather bug was reported upstream on 5 Oct 2026 (section 5.1), and the narrow-output split-K dependence was first found by our own prefill-kernel study (section 5.2).

2. Background

2.1 The M5 GPU and its matrix units

- Apple's term is "a Neural Accelerator in each core" of the M5 GPU; Apple claims "over 4x the peak GPU compute performance compared to M4" for AI workloads (self-reported; Apple Newsroom, 15 Oct 2025). We call these units the matrix units. Metal 4 programs them through the Metal Performance Primitives (matmul2d on cooperative tensors).
- Measured on our MacBook Pro (M5 Pro, 18-core CPU, 20-core GPU, 64 GB, 2 TB SSD): a peak of about 14.0 T MAC/s in 16x32x32 bf16 x bf16 -> fp32 operations, with about 10 % run-to-run spread (the same kernel read 10.5 and 11.7 T MAC/s on two days) [K, D:nax_peak.txt; LT:'M27-P'].
- Memory: a 4 GB streaming probe reads 285-286 GB/s [K, D:bw_probe_k90.txt].

2.2 How MLX 0.32.2 picks a quantized kernel

As read from MLX's `quantized.cpp` at v0.32.2 by our prefill-kernel study [PK, N1]:

- decode widths (S <= 8): qmv / qmv_wide / gather_qmv, on the SIMD units; no matrix units;
- dense calls above the vector limit: `qmm_splitk` with split_k = 512 / ceil(M / 32); when split_k <= 1 (N >= 512 at prefill widths) the call goes to `qmm_t_nax` (bm 32 if M <= 32, else 64; bn 64; bk 64);
- narrow outputs (N = 1 here) stay on split-K, with a split that moves with M;
- the sorted gather of SwitchGLU's prefill path (>= 64 ids) runs `affine_gather_qmm_rhs_nax`, bm 32 if rows / E < 64 else 64. At 16 rows the wide vector kernel declines, and the merged hyper-connection read takes MLX's matrix-unit split-K GEMM [K, LL line 117 'R-09b S4'].

2.3 The model's shapes

- Hidden size 2560. 48 layers: 36 Gated DeltaNet (GDN) layers and 12 attention layers. Four residual streams joined by low-rank hyper-connections (hc 4, rank 320). Per layer: one shared expert (2560 -> 640 -> 2560) with a one-output sigmoid gate, and 512 routed experts of the same shape, 10 per token [the model's config.json on Hugging Face, Qwen/Qwen3.8-Flash-Next].
- Checkpoint quantization: router 8-bit g64; the shared expert, its gate and the routed experts 4-bit g64 [same file]. Our expert store runs at about 3.2 bits per expert weight (ranks 0-1 at 4 bits, ranks 2-9 at 3 bits; [MAIN] section 4.1). Kernel names below say 4-bit or 3-bit for the record format a kernel reads, never for a run.

2.4 Memory

GB = `10^9` bytes of the lifetime peak (ri_lifetime_max_phys_footprint), as the run reports print it; the 45 GB line is 41.9 GiB. A memory line is the engine's own footprint on a 64 GB machine, not an operating-system

limit; a 7.45 GB expert tier held as purgeable memory is not counted in it ([MAIN] section 2).

3. The invariance map (measured)

One row per op class of this model. "Inv." = a row's bits do not depend on how many rows share the call. Sources are in the result table (R-numbers).

op class (MLX 0.32.2, M5)	decode / verify: 1 vs 16 rows	prefill: 1,024 vs 4K-8K rows
quantized matmul, 4-bit / 8-bit, N >= 512	**no**: different kernels below and above the vector limit (R1)	inv. once on `qmm_t_nax`: from M 16-256 by shape; the 8-bit router from M 541 (R30)
quantized matmul, narrow N (1-32)	**no** (R1)	**no**: split-K with an M-dependent split (R17)
MLX's own 16-row call, padded rows	inv. across 1-16 real rows (R3)	-
bf16 matmul, K 2560	**no**	**no**: 210-538 of 4K-8K rows differ (R18)
sorted gather_qmm (routed experts)	inv. as shipped	inv. across bm 32 / 64 and groupings, but **rows left unwritten** past 32,767 rows unless padded (R19)
GDN recurrence (fp32 state, sequential)	inv. (R2)	not tested: prefill keeps GDN in 1,024-token chunks
conv1d (GDN and PLE shapes)	inv., S 2-16 (R21)	-
PLE reductions (gate sum, rms_norm, HC mean)	inv., S 2-16 (R21)	-
SDPA (attention)	**no**: other code at 1 row than inside 16; key-count classes past 512 keys and below 128 (R25)	not tested: prefill keeps attention in 1,024-token chunks
softmax (precise) / argpartition / take / elementwise	inv.	inv. (R16)
sigmoid, compiled graph vs eager	**no**: the compiled form uses a faster exp (section 5.5)	-
matrix-unit fp32 accumulation	deterministic, but matches no fixed fp32 order (R12)	-

How each "no" was handled:

- **Quantized and bf16 matmuls at decode widths:** every call with fewer than 16 rows runs at 16 rows (section 4.1). MLX's own 16-row call is padding-independent, so padding alone restores invariance; our own 16-row kernels then made the padded calls fast.
- **Narrow N at prefill widths:** the only narrow projection in the prefill MoE block, the shared expert's one-output gate, stays on the 1,024-row chunks (section 4.6).
- **bf16 at prefill widths:** no bf16 matmul is batched past the chunk size; the quantized ones are.
- **Sorted gather:** rows padded to a multiple of 64 (section 5.1).
- **Attention:** calls padded to 16 rows, key buckets past 512 keys, length classes below 128 keys (section 4.4).
- **Phase-dependent lowering:** the commit replay runs inside the verify scopes (section 5.5).

4. The kernel set

The set is called REF2 in our sources; its extensions REF2b (attention buckets and classes, commit phase, slot call), REF2-T (rank-tiered experts) and NAX6-N (matrix-unit experts on native records) each became a reference after a same-scores test ([MAIN] section 3.4). A check runs every layer at 16 rows.

4.1 Dense projections: one 16-row shape

- **Pad to 16.** Every quantized or bf16 matmul with fewer than 16 rows runs at 16. MLX's own 16-row call is padding-independent at every dense shape tested: rows 1 / 2 / 5 / 8 / 12, padded, equal the all-real call bitwise [K, R3].
- **qmm16c.** A 16-row 4-bit kernel on the matrix units that multiplies the raw 4-bit codes and applies scales and biases after the MMA. It is row-invariant by construction, and has its own numerics: max error against fp32 0.010 on o_proj vs MLX's 0.023 [K, R6]. The 4-bit dense set takes 14.0 ms per 16-row check vs 35.2 ms for MLX's calls (byte floor 7.1); with it the whole dense part falls from 56.8 to 27.4 ms (floor 11.9) [K, R4, R5].

- **DENSE-N.** The same kernel on weights stored in the matrix unit's fragment order: 8.85 -> 7.39 ms per check (x1.20, 74 % of the byte floor), bitwise equal to qmm16c; the in-place form keeps one copy of the weights [K, LL lines 251, 255].
- **qmm16c8 and QMM32** for the drafter's projections: 8-bit codes (error equal to MLX's 8-bit call, better on down; 1.4-4.8x faster than bf16 per drafter shape), and a 32-row form that reads the weights once and equals two 16-row halves bitwise (-0.44 ms per propose vs the halves) [K, LL lines 282, 349].

4.2 Hyper-connection reads: replicating MLX's own split-K loop

- The merged down + inject GEMM (N 324) is a bit-for-bit replica of MLX's matrix-unit split-K loop, with weight loads issued three steps ahead. It saves 14.68 ms per check against the engine's 24.51 ms per 96 reads, and every projection equals MLX bitwise (96 / 96) [K, R9].
- One simdgroup walking all three K partitions is slower (15.93 / 25.69 ms vs 9.82 per 96 reads): 11 simdgroups are too few [K, LL line 118].
- When a reference already exists, a fast exact kernel can be a replica of the framework's own reduction order rather than a new order that then needs its own certification.

4.3 Routed experts

- **experts_nax**, one 16-row matrix-unit tile per expert: batch-invariant (rows 1 / 2 / 4 / 7 / 8 / 12 alone equal the same rows inside 16), and 2.3x closer to fp32 than MLX's sorted path (max error 0.00061 vs 0.00139). 0.776 vs 1.242 ms per layer at 16 rows, 75 % of the byte floor: 37.2 vs 59.6 ms per check [K, R7, R8].
- **Slot call.** The kernel runs on the expert pool's slots, with no gather into a fresh scratch per layer: -384 ms per 16-row check in the live loop of that day, exact, for checks of up to 16 rows [K, LL line 153].
- **NAX6 / NAX6-N.** A fixed split-K over the matrix units (NAX6), then records stored in the fragments' native order (NAX6-N). In an engine A/B against our 3-bit ALU tier kernel (3c-N), each exact to its own reference, on the 13 prompts kept in all four passes at C 192: whole decode 44.74 vs 42.12 tok/s (x1.062), lifetime peak 38.11 vs 38.10 GB [K, R23]. That screen recorded no prefill wall, so whole request and first token are not available for this A/B (a limit of the 6 Nov harness, not re-run).
- **One-call out-of-order.** Resident experts first, late slots resolved inside the same kernel, with coherent(system) reads of the pool (section 5.7). On 12 IFEval prompts at C 256, two runs each: with coherent reads, 12 / 12 prompts token-identical to one-token decoding in both runs, whole decode 37.8 / 37.9 tok/s, whole request 21.1 / 21.3 tok/s, first token 2.83 s (2.23-3.79) / 2.80 s (2.17-3.73); with plain reads 3 / 12 in both runs, with different divergences, at 37.7 / 38.0 tok/s. Lifetime peak 41.70 GB in all four [K, R14].

4.4 Attention

- Calls with fewer than 16 rows run through the verify path at 16, and the cache is trimmed back [K, R2].
- **Key buckets.** Past 512 keys, every 16-row attention call runs at the next multiple of 512 keys (zero-padded, masked); rows whose one-token step lands in another bucket are recomputed there. With buckets, every logits row past 512 keys equals the bucketed one-token reference: 160 / 160 (a geometry prompt, positions 520-679) and 220 / 220 (HumanEval/68, positions 367-586, across 512); with buckets off, the control breaks (2 / 160) [K, R25].
- **Length classes below 128 keys:** 1,477 / 1,477 tokens and 441 / 441 logits rows equal on 6 prompts [K, R25].
- **One SDPA call per layer** instead of 12 head chunks: -4.06 ms of fixed cost per check (48.40 -> 44.34 ms), bitwise to the reference below 512 keys [K, R26]. Past 512 keys its bits follow a different class split, so it is a FAST option, not part of the exact set [K, LT:'SDPA-1 RESULT'].

4.5 The self-test

`ref2.self_test` checks on random data, for each kernel, that a row computed alone (padded to 16) equals the same row inside a 16-row call: qmm16c (6144 x 2560), MLX's own 16-row call (512 x 2560), the expert kernel (rows alone, and the split form equal to the fused experts_nax), and a bf16 Linear [S:ref2.py, self_test]. The evaluation suite asserts it before every run against an exact reference [ES:evalsuite/live16.py], and we run it after every change to the kernel module. It checks invariance only; exactness against a reference is the audits' job ([MAIN] section 3.2).

4.6 Prefill

- The reference is 1,024-token chunks. Batching a layer's MoE block over all of a prompt's chunks (with the next layers' experts read ahead) is exact only with (a) the sorted rows padded to a multiple of 64 (section 5.1) and (b) the shared expert's one-output gate kept per chunk (section 5.2). Every other op of the block is row-invariant: router, softmax, top-10 selection and normalisation, the padded expert GLU even with 6,216 rows on one expert, the weighted sum, the shared expert and the residual write [K, R15, R16].
- In the engine, with both pinned and up to 8,192 rows per call: decode tokens equal and logits hash equal on 8 / 8, 28 / 28 and 22 / 22 checks for 1.6K / 4.7K / 7.9K-token prompts; prefill 215 / 432 / 500 tok/s vs 189 / 358 / 433 for the chunked path (x1.14 / 1.21 / 1.16); lifetime peak 46.23 vs 45.09 GB [K, R20]. At 2,048 rows per call the peak is 45.81 GB at x1.12-1.17 [K, R20]: both stay over the 45 GB line.

5. MLX findings

5.1 The sorted gather leaves rows unwritten past 32,767 rows

- **Symptom.** SwitchGLU's up projection alone on a 7,932-token prompt (79,320 sorted rows, top-10): 32,776 rows differ between one call and 1,024-token calls; the one-call rows are unwritten [K, R19].
- **Cause.** `affine_gather_qmm_rhs_nax` computes each simdgroup's remaining rows as `min(SM, short(max(0, (M - (y_row + tm)))))` when M is not a multiple of the tile [MLX:quantized_nax.h line 1547]. Past 32,767 remaining rows the `short` wraps negative, and those simdgroups skip compute and store.
- **Workaround.** Pad the sorted rows to a multiple of 64 (BM), which takes the aligned branch: padded one call equals the 1,024-token calls bitwise at 15,650 / 47,300 / 79,320 rows, and runs the routed experts x1.42 / 1.89 / 1.96 faster [K, R19].
- **Upstream.** MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", opened 5 Oct 2026 and closed 6 Oct; the reporter could not reproduce it on MLX 0.32.3, and the thread attributes the fix to PR #3922. It was reported there before we hit it independently [PK, N3; BM: LEDGER.md BUG1]. We stay on 0.32.2 with the padding, because 0.32.4 changes tokens against our exact references.
- Reproduction: S:pfb_diag2.py (the symptom), S:pfb_fix.py (the padded call).

5.2 Narrow-output split-K depends on the row count

The shared expert's gate (4-bit, one output) runs through `qmm_splitk` with an M-dependent split. One call vs 1,024-row calls: equal up to M 1,025, then 41 rows differ at M 1,100, 310 at 1,565, all rows 0-2,047 at 2,048, 4,068 at 8,192. The N 640 projections of the same expert are equal at the same M [K, R17]. On the real layer-0 weights our prefill-kernel study found 4,748 of 8,192 rows differ [PK, N3]. Reproduction: S:pfb_seg.py.

5.3 bf16 matmul is not row-invariant at prefill widths; quantized N >= 512 is

At K 2560, a bf16 matmul over 4K-8K rows differs from 1,024-row calls on 210-538 rows; the 4-bit and 8-bit quantized calls at N >= 512 are equal [K, R18]. Our first version of this test skipped quantized outputs under 32 columns and so missed section 5.2; the correction is in section 7.2. Reproduction: S:pfb_dense_inv.py.

5.4 Attention's key-count classes

- MLX's fused SDPA has exactly one class boundary at 128 keys between 120 and 33,000 keys on the vector kernel [K, LL line 152 'R-40 STEP 3m / 3l'].
- Past about 512 keys, MLX's full attention kernel for 16 queries gives key-length-dependent bits: 2 / 16 rows bitwise, max logit difference up to 4.6 [K, LL line 157 'R-40 STEP 4a'].
- A synthetic test had said SDPA was invariant to the key length (0 of 40 extra-key lengths changed a row); real data broke it [K, LL line 151 'R-40 STEP 3o']. Invariance tests need the real weights and activations.
- Reproduction: S:r40s3o_screen.py, S:r40s3m_screen.py, S:r40s4a_screen.py, S:r40s4c_screen.py.

5.5 Phase-dependent lowering

- The model's attention gate is a sigmoid. Inside a compiled verify graph it is lowered with a faster exp than in eager mode, so the commit's replay outside the verify scopes differed in the last bit. Running

the commit replay inside the verify scopes fixed it: a math prompt that had diverged went to 256 / 256 tokens equal [K, LL line 148 'REF2 commit_phase'].
- Kernels that replay MLX's eager bf16 sigmoid must use `metal::precise::exp`: the JIT default and `fast::exp` each miss 1 of 65,536 bf16 inputs; precise misses 0 [K, LL line 190].

5.6 The matrix unit's fp32 sums match no fixed order

1,024 adversarial tests x 512 outputs: no sequential, pairwise or blocked fp32 order matches the matrix unit's result (best 45-45.6 %). Against the exact sum, 45 % are bitwise, 39 % off by 1 ulp, 14 % by 2-8 ulp and 2.4 % by more. Integer operands match 100 %, and the relaxed-precision flag changes nothing [K, R12]. So an ALU kernel cannot stand in bitwise for a matrix-unit one, and choosing between them by group size or row count breaks row invariance. Reproduction: S:nax_order_probe.py.

5.7 Stale reads of CPU-written memory inside one command buffer

- **Probe.** A 2 MB slot that the GPU has read is rewritten by the CPU, then read again (20 trials each, two runs): the first read after the rewrite is clean (0 / 20 trials stale in both); the same kernel's plain loads return stale words in 20 / 20 and 19 / 20 trials (all 524,288 words in every trial of the first run); a later dispatch in the same command buffer returns 505,808-524,288 and 96-524,288 stale words (20 / 20 in both); adding a buffer memory barrier still leaves 16 / 20 and 20 / 20 trials stale; coherent(system) loads read 0 stale words (0 / 20 in both); a slot last read by an earlier, completed command buffer is clean (0 / 20, second run) [K, R13].
- **Cost of the fix,** rough speed under a share lock. Coherent loads on the pool pointers only (codes, low-bit words, scales, biases): +0.7 % and +1.5 % on NAX6-N's kernels in two runs, +26.3 % on the 3-bit ALU kernel in the one run where its coherent form built, because its single-byte code loads are slow when coherent [K, R13].
- **In the engine.** The one-call path of section 4.3 made exact chat nondeterministic (3 / 12 prompts equal, with different divergences per run) until coherent reads were turned on (12 / 12 twice) [K, R14].

5.8 Streams are thread-local; eval releases the GIL

MLX 0.32.2 releases the GIL in `eval` (a side thread keeps 99-104 % of its solo rate), and streams are thread-local: a second thread must create its own stream, and the main thread must build the graph on it. In that form a two-thread build / encode is bitwise equal to the serial loop over 200 checks [K, LT:'ENC2 + PF-B dense invariance'; D:enc2*.txt]. Its speed is a negative result (section 7).

5.9 fp32 matmuls default to TF32

MLX 0.32.2 sets `MLX_ENABLE_TF32=1` by default. An fp32 matmul then has an error of 4.15e-4 against 6.5e-7 with it off, which broke an fp32 training reference in our drafter study until the flag was set to 0 [K, TR:mtphead line 124].

5.10 A custom kernel's inputs change address space with their size

`mx.fast.metal_kernel` generates the kernel's signature from the arrays it is called with, and an input's address space depends on its size. A prefill kernel of the engine (one that rebuilds pool-resident experts on the GPU) passes inputs it does not need in a call as one-element placeholders. In the run, MLX declared exactly those inputs `const constant uint32_t*`, and the kernel failed to build: "assigning 'const constant uint32_t *' *... to 'const device uint* ' changes address space of pointer". The kernel's unit test had passed because it called the kernel with full-size arrays only. A 1,024-element placeholder is declared `device`; with it the same run completed, with tokens equal to the path without the kernel on 12 / 12 prompts and logits hashes equal at 721 / 721 checks [K, R32]. The workaround is a placeholder that is not small, and a unit test that calls the kernel with the smallest inputs the engine passes (the engine's test now does) [K, R32]. MLX's custom-kernel guide shows generated signatures with `device` inputs only and does not mention the size rule (opened 9 Nov); we did not locate the rule's size cut in MLX's source.

6. GPU and matrix-unit measurements

All on one MacBook Pro; "quiet" = no other GPU job. Kernel times are from dependent chains of the engine's own calls; the per-call synchronised timings swing 2-3x between processes and are not used [K, LT:'STEP 0

PART B RESULT'].

6.1 Peak and arithmetic

About 14.0 T MAC/s at the matrix units (section 2.1, R11); their fp32 accumulation order (section 5.6, R12).

6.2 Streaming efficiency of the expert kernels

At S 16, quiet, a loads-only replica of NAX6-N's memory access reads 295-296 GB/s, the machine's streaming rate. NAX6-N itself runs at 74 % of it (218-219 GB/s), a mixed FAST kernel at 79-81 %, and NAX6 at 58-59 %; across S 8-32 the replica reads 287-305 GB/s and NAX6-N runs at 70-74 % [K, R24]. The gap is exposed compute, not bandwidth: making an earlier expert kernel bandwidth-bound did not pay (section 7).

6.3 One dependent dispatch costs about 1.6 us

A chain of 200 dependent kernels that move almost no bytes: 1.24 us per kernel at 32 threads, 1.40-1.56 us at 1,024 to 20,480 threads, 4.13 us when 1 MB moves [K, R27]. A check launches about 1,000-4,000 kernels, so dispatch is worth about 1-6 ms per check (1,000-4,000 x 1.24-1.56 us), not the 20 ms an earlier fit assumed [P, R27].

6.4 The dense time is lost in narrow matrix-vector kernels

Per call in a dependent chain, at 2 / 16 rows [K, R27]. MB = `10^6` bytes the call reads and writes (weights, scales and biases, its input and output; the raw report counts them in KiB); GB/s = MB / us, at 2 and 16 rows:

kernel	tiles	us per call	MB per call	GB/s	calls per check
GDN input projection	512	113 / 115	24.2	210-213	36
attention query projection	384	75 / 74	18.2	243-247	12
GDN / attention output projection (K 6,144)	80	112 / 112	9.1	81-82	48
shared expert gate / up	41	48 / 47	2.0	42-43	48
attention key and value projections	16 each	48 / 49	0.84	17-18	24
GDN a / b projections	3	45 / 46	0.22	5	36

- Wide kernels already stream near the machine's rate. The narrow ones walk the whole K dimension with one simdgroup per 32-column tile, so with 3-80 tiles the GPU is mostly empty and the call costs about 1.1-1.2 us per K group whatever its bytes [K, R27].
- **Split K per shape (DN-KS).** Enough simdgroups per call: output projection 114.8 -> 41.4 us (214 GB/s on its 8.85 MB of weights), a / b 47.3 -> 10.4, key / value 47.4 -> 9.4, shared gate / up 48.4 -> 14.7, shared down 16.7 -> 9.1; -7.74 ms per 2-row check over these five in the microbenchmark and -4.19 ms of fixed cost in the engine (55.17 / 55.30 -> 51.05 ms) [K, R28]. The split changes the summation order, so it is not bitwise to the reference; it stays row-invariant, so a reference built with it would again be exact to itself, after its own certification, which we did not run (section 9).

6.5 Where a check's time goes

- **Fixed cost** = the steady 2-row cycle minus twice the warm per-row slope: 53.74 ms with the 6 Nov fast stack, against a pre-registered PASS line of 30 ms (KILL above 36) [K, R29]. The per-row slope is 3.01 ms per row on the expert kernel of that day [K, R29].
- **Per piece, a warm 2-row check, two measures of the same run** [K, R29; F:2026-11-06_0140_ flat2-fixed-cost-per-check]:

piece	wall time, one sync per piece (ms)	GPU time (ms)
GDN	15.7	13.2
routed experts	17.4	11.1 (about 4.6 of it fixed)
hyper-connection reads	11.0	8.6
attention	8.6	6.4
router + shared expert	5.7	5.7
MoE tail	3.5	1.1

piece	wall time, one sync per piece (ms)	GPU time (ms)
sum	111.1 (all pieces, incl. embed / norm / head 2.3)	47.8 (all pieces, incl. HC write 0.8, PLE 0.65)

The wall column pays a fresh command-buffer start per piece, so read it as proportions, not time. The GPU column is each piece's synced time minus the 0.133 ms round trip per sync; its sum is about the whole unsynchronised check, so the check is GPU-bound and the 13.4 ms CPU graph build is hidden behind GPU work. Only the experts grow with rows: 11.1 ms of GPU time at 2 rows, 57.0 ms at 16 with all experts resident [K, same source].

- At about 12 tokens kept per check, a 54 ms fixed cost alone caps code near 220 tok/s even with free experts [P, from R29].

6.6 Prefill throughput

MLX's 4-bit qmm runs at about 27-29 TF from 1K rows; the routed experts' gather_qmm rises from 7.1 TF at 1K rows to 13.8 at 8K [K, R30]. In the batched prefill MoE call (section 4.6) the routed experts run at 11.2 TF on a 7.9K-token prompt vs 5.7 TF in 1,024-token calls [K, R19].

7. Negative results

7.1 Kernels that did not pay (each exact unless marked)

idea	result	verdict	source
int4 operands straight into the matrix units (Metal 4 matmul2d, bf16 x uint4)	bitwise and row-invariant, but slower: 16-row experts 0.992 vs 0.757 ms per layer (58 % of floor); dense x0.68-0.87	KILL	R10; LL line 72
software-pipelined expert kernel (NAX7)	11 % slower (96 / 96 bitwise)	KILL	LL line 219
loader / compute simdgroup specialisation (NAX7-WS)	35 % slower	KILL	LL line 220
the expert kernel as three dispatches (NAX7-F)	+1.2 %, one unexplained hang at S 32 under load	KILL	LL line 223
persistent expert kernels (NAX8-P1)	best configuration -5.2 % at S 16; slower in every residency	KILL	LL line 306
3-bit code decode by table lookup	device table -1.1 %, threadgroup table -15.1 % at S 16	KILL	LL line 312
int8 activations x int4 codes, int32 sums (NAX-C; FAST)	same speed as NAX6 (-0.3 % at S 16), error 1.07e-2 x RMS; on native records -1.7 %	KILL	LL lines 226, 305
making the expert kernels bandwidth-bound (K-60)	the headroom was not bandwidth	KILL	LL line 168
two-thread graph build / encode (ENC2)	bitwise, but x0.96 on its toy	KILL	LL line 389
GEMV on native records for 1-2-row checks (NGEMV; FAST)	-1.41 / -0.80 ms per check in isolation; no engine gain, the chat check is CPU-bound	KILL as speed	LL lines 384, 388
split-K drafter attention (DSDPA)	x0.56, then x0.51 at 2,054 keys against a x0.50 line	missed	LL lines 304, 326
trellis-coded experts on the matrix units (TRL-NAX; FAST)	0.87x NAX6-N's time at S 16, but the trellis store failed the quality screen	not built	LL lines 322, 325
the Neural Engine beside the GPU's expert kernel	per-layer GPU -> ANE -> GPU handoff 78-147 us	KILL	LL line 243
one simdgroup per HC read (R-09b F4 / F8)	15.93 / 25.69 vs 9.82 ms per 96 reads	KILL	LL line 118
GPU timeline from a Metal System Trace	the trace streams counters to the temp dir at about 120 MB/s; replaced by knock-outs and dependent chains	method KILL	LL line 97

7.2 Results we corrected

- **"Batched prefill is row-invariant at the MoE block's dense shapes."** Our first test passed, but it skipped quantized outputs under 32 columns, the shared expert's gate among them. The engine's batched prefill then broke the logits hash; the gate's split-K was the cause (section 5.2), and keeping it per chunk made the prefill exact [K, R15, R16; LT:'PF-B EXACTNESS', CORRECTION].
- **"SDPA is invariant to the key length"** held on synthetic data and failed on real data (section 5.4).
- Across the paper set, three headlines were withdrawn or corrected: "12 GB, 40.8 tok/s 'exact'" (a FAST arm; withdrawn); "45 GB FAST stack, 157.9 tok/s" (included CD16, which failed MBPP; withdrawn); "180.3 tok/s" was the registered steady metric, which is 142.8 on whole decode, pooled (metric corrected 7 Nov; the correction note's 142.1 is a geo mean). None is a kernel result; [MAIN] section 10.0 has them.

8. Related work

- **Batch invariance.** Thinking Machines Lab (H. He et al., "Defeating Nondeterminism in LLM Inference", 10 Sep 2025) trace inference nondeterminism to kernels that are not batch-invariant and make RMSNorm, matmul and attention invariant by keeping each element's reduction order fixed whatever the batch; attention uses fixed-size KV splits rather than a fixed number of splits. Their code (batch-invariant-ops) runs in vLLM's deterministic mode; on their Qwen-3-8B benchmark default vLLM took 26 s, the deterministic mode 55 s, and 42 s with an improved attention kernel (self-reported). Our key buckets (section 4.4) are the same idea as fixed-size KV splits. What differs here is the consumer (a 16-row speculative check against one-token decoding, not a serving batch), the hardware (Metal, the M5's matrix units, whose fp32 order no ALU kernel can match, section 5.6), and that the reference is fixed first and the fast kernels are made to reproduce it, sometimes by replicating the framework's own loop (section 4.2).
- **Deterministic serving modes.** vLLM has an opt-in batch-invariance mode (`VLLM_BATCH_INVARIANT=1`; its documentation lists NVIDIA GPUs only) and SGLang a deterministic mode (`-enable-deterministic-inference`) that integrates the Thinking Machines kernels, with attention at a fixed split-KV size; both say the mode costs speed (vLLM documentation, "batch invariance"; LMSYS blog, "Towards Deterministic Inference in SGLang and Reproducible RL Training", 22 Sep 2025).
- **Determinism through verification.** LLM-42 (Gond, Kamath, Ramjee and Panwar, arXiv 2601.17768, Jan 2026) decodes on a nondeterministic fast path and replays the candidate tokens under a fixed-shape reduction schedule, committing the consistent ones and rolling back the rest; it argues that batch-invariant kernels impose a fixed cost even when little traffic needs determinism. Our check is the same pattern with one difference: the reference itself is defined at the verify's fixed 16-row shape (one-token decoding runs through the same padded kernels, section 4.1), so the fast path that drafts can be any drafter and the cost of exactness is the x0.75 of one-token decoding (R31), paid back by the tokens each check keeps.
- **Speculative decoding.** Leviathan et al. (ICML 2023; arXiv 2211.17192) and Chen et al. (2023; arXiv 2302.01318) define draft-then-verify decoding that keeps the target model's distribution; with greedy decoding, exactness further needs the verify pass to reproduce one-token decoding's bits, which is this paper's subject. Our drafters are our fine-tune of DFlash2 (Inco AI; Apache-2.0), a block drafter of the DFlash method (Chen, Liang and Liu, arXiv 2602.06036; code z-lab, MIT), and the model's MTP head ([DRAFTER]).
- **Frameworks.** MLX (Apple; MIT), and MTPLX by Youssof Altoukhi (https://github.com/youssofal/mtplx; Apache-2.0), whose model code for this architecture and MTP decoding loop our engine builds on. The sorted-gather bug was reported upstream as MLX issue #4632 (section 5.1).
- **Same model, same machine.** Strata (MIT; github.com/Niko1221/Strata), from our own runs ([MAIN] section 9 and Appendix G). Published runs of this model on other machines are in [MAIN] Appendix H (self-reported).

9. What we left on the table

- **GPU idle time.** During chat decode the GPU is busy 81 % of the time on average (p10 74 %, p90 86 %); in-kernel spin counts as busy, so the idle share is at least 19 % [K, R22].
- **Exposed compute in the expert kernel.** NAX6-N runs at 70-74 % of the rate its own loads stream at; the rest, 26-30 %, is exposed compute [K, R24].
- **A non-bitwise split-K for the narrow projections** saves 4.19 ms of fixed cost per check in the engine; it needs a new reference and its certification, which we did not run [K, R28].
- **The chat check's CPU side.** NGEMV's -1.41 / -0.80 ms per check (FAST) showed no engine gain because the chat check is CPU-bound: about 31.5 ms of graph build in a 33 ms check [K, LL line 388]. Every GPU-side saving on chat waits for a shorter build.
- **The prefill peak.** The exact batched prefill gains x1.14-1.21 but peaks at 45.8-46.2 GB, over the 45 GB line; a variant under the line was not found [K, R20].
- **Prefill overlap.** At 7.9K tokens the expert reads overlap compute for about 1.2 of about 6 s, and the batching delivered about half of its kernel-level estimate [P, from D:pfb_bigmoe_report.txt].
- **Matrix units at verify widths.** MLX runs no matrix units below its vector limit (S <= 8) [PK, N1]; our 16-row kernels fill that gap only for the shapes we wrote.

Result table (sources in Appendix A)

#	result	exact? (to which reference)	number	mark	source
R1	stock kernels: 16 one-token steps vs one 16-row verify	-	all 16 rows differ at record 0 (layer 0 GDN matmul); logits 0 / 16	[K]	D:inv0_screen.txt; LT:'R-40 REF2'
R2	the REF2 kernel set, same test	exact to REF2	16 rows x 37 records bit-identical, 12-layer slice, 1K context	[K]	D:inv1_screen.txt
R3	MLX's 16-row call, rows 1 / 2 / 5 / 8 / 12 padded	inv.	bitwise equal to the all-real call, every dense shape	[K]	D:kern23.txt
R4	qmm16c, the 4-bit dense set per 16-row check	exact (REF2)	14.0 vs 35.2 ms (floor 7.1)	[K]	D:kern23.txt
R5	dense part per 16-row check	exact (REF2)	56.8 -> 27.4 ms (floor 11.9)	[K]	D:kern23.txt
R6	qmm16c accuracy vs fp32, o_proj	-	max error 0.010 vs MLX 0.023	[K]	D:kern23.txt
R7	experts_nax per layer, 16 rows	exact (REF2)	0.776 vs 1.242 ms (75 % of floor); 37.2 vs 59.6 ms per check	[K]	D:qmmg25.txt
R8	experts_nax accuracy vs fp32	-	0.00061 vs 0.00139 (2.3x)	[K]	D:qmmg25.txt
R9	HC read as a replica of MLX's split-K loop	bitwise = MLX (96 / 96)	-14.68 ms per check vs 24.51	[K]	D:r09c_screen.txt
R10	int4 operands into the matrix units	bitwise, inv.	16-row experts 0.992 vs 0.757 ms per layer; in_proj_qkv 3.05 vs 2.60; o_proj 3.41 vs 2.31; in_proj_z 2.26 vs 2.07 ms (KILL)	[K]	D:qmm27.txt; LT:'L1-27'
R11	matrix-unit peak, 16x32x32 bf16 -> fp32	-	~14.0 T MAC/s; run-to-run ~10 %	[K]	D:nax_peak.txt
R12	matrix-unit fp32 sum vs any fp32 order	-	no order matches (best 45-45.6 %); vs the exact sum 45 / 39 / 14 / 2.4 %	[K]	D:nax_order_probe.txt, nax_order_fit.txt
R13	stale CPU-written slot inside one command buffer (2 MB, 20 trials)	correctness	two runs: B0 control 0 / 20 (both); same kernel stale 20 / 20 and 19 / 20; next dispatch 505,808-524,288 and 96-524,288 words, 20 / 20 (both); + memory barrier 16 / 20 and 20 / 20; coherent(system) 0 / 20 (both); earlier command buffer 0 / 20 (run 2). Cost, rough speed (share lock): NAX6-N +0.7 % and +1.5 % (two runs), 3c-N +26.3 % (one run)	[K]	D:s7b_probe.txt, s7b_probe2.txt, coh_check.txt; LT:'S7B-PROBE', 'COH'
R14	coherent one-call in the engine (12 IFEval, C 256, two runs each)	exact (NAX6-N one-call)	coherent: == one-token decoding 12 / 12 both runs, whole decode 37.8 / 37.9, whole request 21.1 / 21.3 tok/s, first token 2.83 (2.23-3.79) / 2.80 (2.17-3.73) s; plain: 3 / 12 both runs, 37.7 / 38.0 and 20.0 / 20.1 tok/s; lifetime peak 41.70 GB	[K]	D:chs4/chs4_report.txt lines 21-24; whole request and first token pooled from BM:data/r57/ycoh{,2}_256_runs.json, ycoc{,2}_256_runs.json
R15	prefill MoE block, chunked vs batched call shapes	exact to chunked prefill	only the shared-expert gate differs (304 / 1,565; 2,326 / 4,730; 4,451 / 7,932 rows)	[K]	D:pfb_moe_inv.txt
R16	the same with the gate per chunk	exact	all ops equal at 6 shapes	[K]	D:pfb_seg.txt
R17	narrow-N split-K, one call vs 1,024-row calls	**not** inv.	equal to M 1,025; 41 rows differ at 1,100, 310 at 1,565, rows 0-2,047 at 2,048, 4,068 at 8,192; N 640 control equal	[K]	D:pfb_seg.txt; PK N3
R18	bf16 vs q4 / q8 matmul at 4K-8K rows	q inv., bf16 not	bf16 210-538 rows differ; q4 / q8 equal	[K]	D:pfb_dense_inv.txt

#	result	exact? (to which reference)	number	mark	source
R19	sorted-gather bug	correctness	79,320 sorted rows: 32,776 differ (unwritten); padded to 64: one call == 1,024-token calls at 15,650 / 47,300 / 79,320 rows, experts 31.2 -> 22.0, 80.4 -> 42.6, 136.3 -> 69.4 ms (5.7 -> 11.2 TF at 7.9K)	[K]	D:pfb_diag2.txt, pfb_fix.txt
R20	batched prefill MoE + read-ahead in the engine, gate per chunk	exact to chunked prefill	tokens == and logits hash 8 / 8, 28 / 28, 22 / 22; 215 / 432 / 500 vs 189 / 358 / 433 tok/s (1.6K / 4.7K / 7.9K); peak 46.23 vs 45.09 GB; at 2,048 rows per call 212 / 418 / 486 tok/s, 45.81 GB	[K]	BM:data/pfd/pfd_report.txt at commit a6d5347, part B (it rounds peaks to 0.1 GB); peaks from BM:data/r57/{pdb38s,pdpa1,pdb32s}_256_runs.json (mem)
R21	conv1d and PLE reductions at S 2-16	inv.	conv1d: 0 of 64 rows (0 of 655,360 elements) differ at every S, both shapes; PLE reductions: 0 differing rows for every op at every S	[K]	D:r40s3s_screen.txt; LT:'R-40 STEP 3s', '3r'
R22	GPU busy during chat decode (C 256)	-	mean 81 %, p10 74, p90 86, min 72, max 88 (40 x 0.5 s; in-kernel spin counts as busy)	[K]	D:gpu_util_m1b.txt
R23	NAX6-N vs the 3-bit ALU kernel (3c-N), engine A/B, C 192, quiet, 13 prompts kept in all 4 passes	exact (each to its own reference)	whole decode 44.74 vs 42.12 tok/s (x1.062; the steady-state rate, information only, read x1.0665); lifetime peak 38.11 vs 38.10 GB; whole request and first token not recorded by that screen	[K]	D:natfri/r57/nf*_192_runs.json via S:natfri_pooled.py -> D:natfri/natfri_pooled.txt; LT:'NATIVE-FRI' RUN 2
R24	expert kernels vs the machine's streaming rate (S 8 / 16 / 32, quiet)	-	4 GB probe 285-286 GB/s; loads-only replica of NAX6-N 287-305 (S 16: 295-296); NAX6-N 70-74 % (S 16: 74 %, 218-219 GB/s); FAST hybrid 76-81 % (S 16: 79-81 %); NAX6 55-59 % (S 16: 58-59 %)	[K]	D:k90_arm012.txt, bw_probe_k90.txt; LL line 311
R25	key buckets past 512 keys; SDPA classes below 128 keys	exact (REF2b)	buckets 160 / 160 and 220 / 220 rows; buckets off 2 / 160; classes 1,477 / 1,477 tokens and 441 / 441 logits rows on 6 prompts	[K]	D:r40s4c_screen.txt; LT:'R-40 STEP 4c'; TABLE_350 line 2980
R26	one SDPA call per layer	bitwise below 512 keys (FAST past it)	fixed cost 48.40 -> 44.34 ms (-4.06)	[K]	D:stepb/out/stepb_{b,batt,b2}*224_sdpa**; LT:'SDPA-1 RESULT'; LL line 345
R27	dependent dispatch and narrow-kernel costs	-	1.24-1.56 us per dependent kernel (4.13 with 1 MB); per-kernel table of section 6.4	[K]	LT:'STEP 0 PART B RESULT'; D:stepb/
R28	split K per shape (DN-KS)	not bitwise (new order, row-invariant)	-7.74 ms per 2-row check (micro); engine fixed cost 55.17 / 55.30 -> 51.05 ms (-4.19)	[K]	D:dnks_bench.txt; D:stepb/out/stepb_{fri,friks,fri2}*224_ab**; LT:'DN-KS step 1 / 2 RESULT'
R29	fixed cost of a check (FLAT-2)	-	53.74 ms (PASS line 30, KILL > 36); slope 3.01 ms per row	[K]	LT:'FLAT-2 RESULT'
R30	MLX prefill kernels and row invariance by M	-	4-bit qmm ~27-29 TF from 1K rows; gather_qmm 7.1 TF (1K) -> 13.8 (8K); qmm row-invariant from M 16-256 by shape, the 8-bit router from M 541	[K]	LL line 380; LT:'PREFILL 500'
R31	one-token decoding through the padded kernels	exact (REF2)	11.65 vs 15.50 tok/s over 210 rows (x0.752; row time, prefill included; information only in that report); peak 35.6 GB	[K]	EV:RESULT_ref2_2026-10-28, Speed table; LT:'R-40 REF2' step 2

#	result	exact? (to which reference)	number	mark	source
R32	custom-kernel inputs by size (section 5.10)	exact	one-element placeholder inputs declared `const constant`: build error in the run; a (1, 1024) uint32 placeholder builds; the re-run's tokens equal the path without the kernel 12 / 12, logits hash 721 / 721; unit test with the engine's placeholders PASS	[K]	BM:data/pfd/pdz4a_256.txt at commit 3175ef3 lines 71-80 (the error); BM commit 9aa398b (the fix); BM:data/pfd/pfd_report_0009.txt line 16 (pdz4a); BM:scripts/pfd_unit.py line 147

Appendix A. Sources

Folder paths appear only here. Paths are relative to the root of the public snapshot, github.com/kamenlevi/350-tps; commit hashes are those of each folder's own repository, and the snapshot holds the files as last committed.

key	file
LT:'*heading*'	a section of `Leap/TABLE_350.md` (the kernels ledger)
LL line *n*	line *n* of `FableGod/TRIED/leap.md` (one line per idea, with its verdict)
TR:mtphead line *n*	line *n* of `FableGod/TRIED/mtphead.md` (the drafter study's ledger)
D:*file*	`Leap/data/<file>` (raw output)
S:*file*	`Leap/scripts/<file>` (the test that produced it)
PK	`PrefillKernels/FINDINGS.md` (the prefill-kernel study)
BM:*path*	`BigMoE/<path>` (the engine; "at commit c" = the file at the engine repository's commit c)
EV:*name*	`EvalSuite/docs/<name>.md` (the evaluation suite's results)
ES:*file*	`EvalSuite/<file>` (the evaluation suite's code)
MLX:*file*	`mlx/include/mlx/backend/metal/kernels/<file>` in the installed MLX 0.32.2 package
F:*name*	`FableGod/findings/<name>.md` (dated run notes)
[MAIN], [EVAL], [DRAFTER], [RELEASE]	the companion papers

R14's whole request and first token were pooled on 8 Nov from the four runs' records (n / (sum of prefill walls + sum of n / tok/s)); the same arithmetic as S:natfri_pooled.py.

Appendix B. Reproduction

- Every [K] row names its raw output (D:) and, in sections 5 and 7, its script (S:). The kernels and tests are released under GPL-3 with the engine ([RELEASE]).
- Software: MLX pinned to 0.32.2 with the pad-to-64 fix for MLX issue #4632 (sorted gather_qmm past 32,767 rows when M is not a multiple of 64); 0.32.4 changes tokens against our exact references.

Appendix C. Limits stated instead of numbers

- GDN and attention at prefill widths: not tested, because the prefill keeps both in 1,024-token chunks (section 3).
- NAX6-N vs 3c-N: whole request and first token were not recorded by the 6 Nov screen (R23).

Use of AI tools

The code, the experiments and the text of this paper were produced with substantial help from AI coding agents. The author directed the work and is responsible for its content. The numbers were re-derived from the raw run records named beside them, and the references were opened by hand or by script.

References

- Youssof Altoukhi. MTPLX (software, v2.12.2, commit 9882703f; Apache-2.0). https://github.com/youssofal/mtplx
- Apple. "Apple unleashes M5, the next big leap in AI performance for Apple silicon." Apple Newsroom, 15 Oct 2025. https://www.apple.com/newsroom/2025/10/apple-unleashes-m5-the-next-big-leap-in-ai-performance-for-apple-silicon/
- Charlie Chen et al. "Accelerating Large Language Model Decoding with Speculative Sampling." arXiv:2302.01318, 2023.
- Jian Chen, Yesheng Liang and Zhijian Liu. "DFlash: Block Diffusion for Flash Speculative Decoding." arXiv:2602.06036, 2026. ICML 2026. Code: https://github.com/z-lab/dflash (MIT).
- Raja Gond et al. "LLM-42: Enabling Determinism in LLM Inference with Verified Speculation." arXiv:2601.17768, 2026.
- Awni Hannun, Jagrit Digani, Angelos Katharopoulos and Ronan Collobert. "MLX: Efficient and flexible machine learning on Apple silicon" (software, 2023; MIT). https://github.com/ml-explore/mlx
- Horace He and Thinking Machines Lab. "Defeating Nondeterminism in LLM Inference." Thinking Machines Lab: Connectionism, 10 Sep 2025. doi:10.64434/tml.20250910. https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/
- Inco AI. "DFlash 2: Keep Drafting Parallel." Blog, Aug 2026. https://inco.ai/blog/dflash2
- Inco AI. Qwen3.8-Flash-Next-DFlash2 (drafter checkpoint, revision a22a174, 19 Oct 2026; Apache-2.0). https://huggingface.co/incoai/Qwen3.8-Flash-Next-DFlash2
- Yaniv Leviathan, Matan Kalman and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding." arXiv:2211.17192, 2022. ICML 2023.
- LMSYS Org. "Towards Deterministic Inference in SGLang and Reproducible RL Training." LMSYS blog, 22 Sep 2025. https://lmsys.org/blog/2025-09-22-sglang-deterministic/
- MLX issue #4632, "[BUG] gather_qmm(sorted_indices=True) returns wrong results on M5 Max (macOS 27) when rows > 32768 and not a multiple of 64", 5 Oct 2026. https://github.com/ml-explore/mlx/issues/4632
- MLX pull request #3922, "Fix sorted gather_qmm NAX row overflow above 32K", merged 26 Aug 2026. https://github.com/ml-explore/mlx/pull/3922
- Niko1221 and the Strata contributors. Strata (software, version 0.1.58-metal, commit 146df3c; MIT). https://github.com/Niko1221/Strata
- Qwen Team. Qwen3.8-Flash-Next (model weights; Qwen Community License 1.0). https://huggingface.co/Qwen/Qwen3.8-Flash-Next
- Thinking Machines Lab. batch_invariant_ops (software; MIT). https://github.com/thinking-machines-lab/batch_invariant_ops
- vLLM project. "Batch Invariance." vLLM documentation. https://docs.vllm.ai/en/latest/features/batch_invariance/

Companion papers:

- [DRAFTER] Kamen Levi. Drafting for a 125B mixture-of-experts model that does not fit in RAM: training its own MTP head, one drafter per text type, and the levers that failed. 2026. A companion paper of this set.
- [EVAL] Kamen Levi. Certifying lossy decoding on the model's known benchmarks: same-scores gates for a 125B mixture-of-experts model on one 64 GB Mac. 2026. A companion paper of this set.
- [MAIN] Kamen Levi. What It Takes to Run a 125B LLM Fast on a 64 GB Mac, and How to Go Faster. 2026. A companion paper of this set.
- [RELEASE] Kamen Levi. Running a 125B mixture-of-experts model on a 64 GB Mac: the engine package. 2026. A companion paper of this set.